



Production RAG

AICB-P2T3 · Ngày 18 · Chương 4 — Agent Nâng Cao

M.Sc Trần Minh Tú

VinUniversity · Phase 2 · Track 3 · Tuần 4

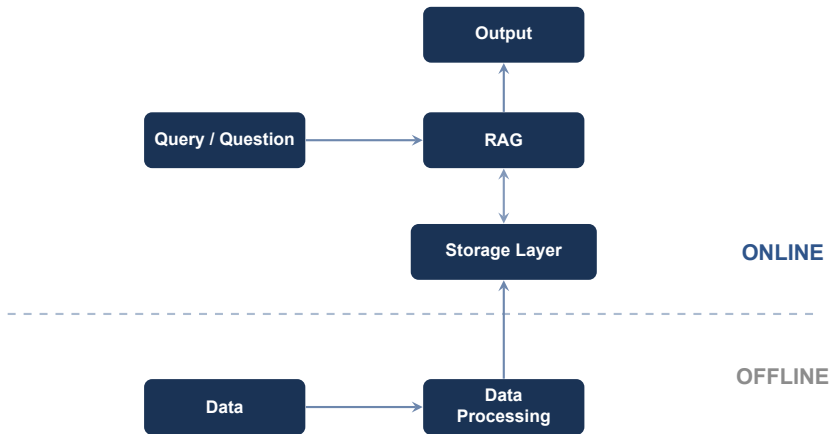
HÃY SUY NGHĨ...

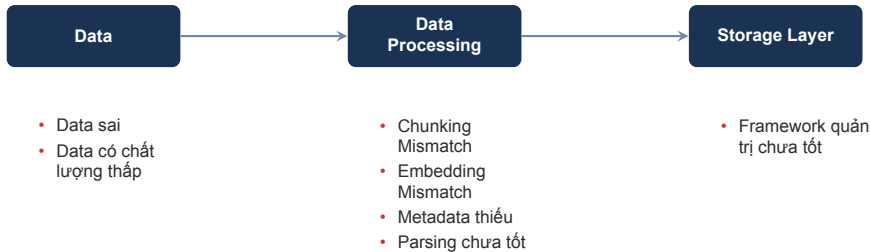
*“Tại sao RAG pipeline demo chạy tốt
nhưng production accuracy chỉ đạt 60%
— ingestion hay retrieval đang giết bạn?”*

Giữ câu hỏi này trong đầu khi học bài hôm nay

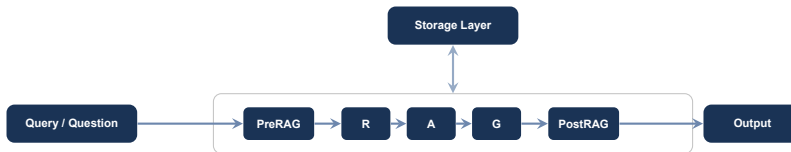
Tại sao Basic RAG thất bại?

Ingestion & Retrieval — failure nằm ở đâu trong pipeline?



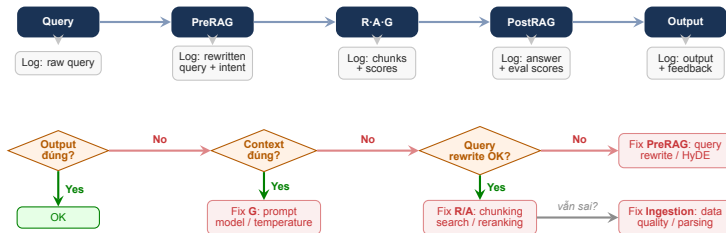


Lưu ý: Trong thực tế: chỉ có ingestion pipeline là chưa đủ. **Enrichment Pipeline** sẽ giúp chúng ta làm giàu thêm thông tin (contextual embeddings, metadata extraction, data cleaning).



Bước	Vai trò	Nguyên nhân output sai
PreRAG	Xử lý query	Query mơ hồ · Không rewrite · Vocabulary gap · Thiếu input guardrails
R	Tìm chunks	Chỉ 1 method · BM25 miss synonyms · Dense miss keywords · Thiếu metadata filter
A	Ghép context	Quá nhiều chunks · Lost in the middle · Context overflow · Thiếu reranking
G	LLM trả lời	Hallucinate · Prompt yếu · Temperature cao · Model không phù hợp
PostRAG	Validate output	Output không qua kiểm duyệt · Thiếu evaluation pipeline · Không có monitoring/feedback loop

Lưu ý: Mỗi bước đều có thể khiến output sai. Production RAG cần fix **toàn bộ chuỗi**, không chỉ 1 bước.



1. Log mọi bước (query, chunks, scores, answer, feedback) → 2. Output sai? Đi ngược: PostRAG → R·A·G → PreRAG → Data → 3. Dừng ở bước đầu tiên có vấn đề → fix đúng chỗ.

60%

Naive RAG
Accuracy

85%+

Production RAG
Accuracy

+25%

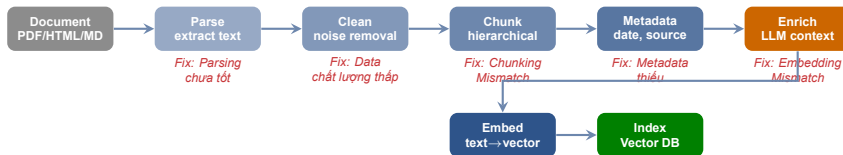
Improvement
khi optimize

Metric	Naive RAG	Production RAG	Nguyên nhân cải thiện
Faithfulness	~0.70	≥ 0.85	Better prompt + reranking
Context Recall	~0.55	≥ 0.75	Hybrid search + enrichment
Context Precision	~0.50	≥ 0.75	Reranking + metadata filter
Answer Relevancy	~0.65	≥ 0.80	Query rewrite + augmentation

Fix OFFLINE — Ingestion Pipeline

Data Processing: Chunking, Embedding & Enrichment

02



Slide 1.2 liệt kê 5 OFFLINE failures.
Pipeline này fix **từng failure một**:
Parse → Clean → Chunk → Metadata
→ Enrich.

Bỏ bước nào = để lọt failure đó vào
Vector DB.

Lưu ý: “Garbage in, garbage out”
— mỗi bước bỏ qua sẽ **tích lũy lỗi**.
Parse sai → chunk sai → embed sai
→ retrieve sai → output sai.

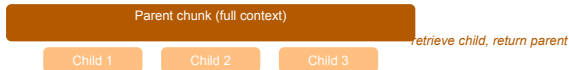
Fixed-size



Semantic



Hierarchical



Hierarchical (parent-child) nên là **default**: chunks nhỏ cho retrieval precision + chunks lớn cho LLM context.

Fixed: 512 tokens, overlap 64
Semantic: cosine threshold 0.85
Hierarchical: parent 2048, child 256

Parse **markdown headers**, **HTML tags**, **PDF sections** rồi chunk theo logical structure.

Giữ nguyên tables, code blocks, lists — không cắt giữa chừng.

Ưu tiên khi corpus có **structured documents** (docs, API refs, manuals).

Dùng **long-context embedding model**, chạy qua toàn bộ document → thu được **token-level embeddings** có full context.

Pool token embeddings theo chunk boundaries → mỗi chunk embedding mang context của cả document.

Khác naive: embed từng chunk riêng lẻ → “orphan”, không biết xung quanh nói gì.

Yêu cầu: jina-embeddings-v2, nomic-embed.

Trade-off: tốn memory + latency cao hơn khi indexing.

Model	Dims	Tiếng Việt	Max Tokens	Cost
text-embedding-3-small	1536	OK	8191	\$0.02/1M
text-embedding-3-large	3072	Tốt	8191	\$0.13/1M
Cohere embed-v3	1024	Tốt	512	\$0.10/1M
bge-m3 (open-source)	1024	Rất tốt	8192	Free
multilingual-e5-large	1024	Rất tốt	512	Free

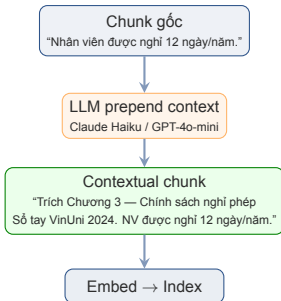
Tiếng Việt → bge-m3 hoặc multilingual-e5-large

Budget có → text-embedding-3-large

Production → Cohere embed-v3 (built-in types)

Lưu ý: Đổi embedding model = **re-index toàn bộ**. Chọn kỹ từ đầu! Benchmark trên MTEB multilingual leaderboard.

Note: Cohere Embed v4 đã hỗ trợ 128K tokens — cân nhắc nếu cần long-context.



Ý tưởng (Anthropic, Sep 2024) — Trước khi embed mỗi chunk, dùng LLM prepend 1 đoạn context ngắn giải thích chunk nằm ở đâu trong document.

Retrieval failure giảm **49%** (alone)
Giảm **67%** khi kết hợp Contextual BM25 + Reranking

Lưu ý: Trade-off: +1 LLM call/chunk khi indexing (one-time). Dùng model rẻ (Haiku, GPT-4o-mini).

PDF chứa charts, tables as images, diagrams.
Text-only RAG bỏ qua **30–40%** info.
Cần embed visual content vào cùng vector space.

Model	Type	Note
CLIP / SigLIP	Text+Img	Baseline
Jina CLIP v2	Text+Img	Multilingual
ColPali/ColQwen2	Full page	Doc-native
Voyage MM 3	Text+Img	API-based

1. Describe then Embed: Image $\rightarrow$ VLM $\rightarrow$ text $\rightarrow$ embed.

+ Dùng existing pipeline. – Mất visual nuance.

2. Native Multimodal: Image+Text $\rightarrow$ shared vector space.

+ Giữ visual info. – Alignment chưa perfect.

3. Document-as-Image (ColPali): Render page $\rightarrow$ embed image. Bypass OCR.

+ Giữ layout/tables. – Tốn storage, cần GPU.

Lưu ý: Chỉ cần khi corpus có $>20\%$ visual content. Pure text $\rightarrow$ text-only đủ. Text-to-image retrieval accuracy thấp hơn text-to-text 15–20%.

Enrichment Pipeline

Làm giàu chunks trước khi embed — Summarize, HyQA, Meta-data

03



LLM-powered (one-time, offline)

Raw chunks thiếu context → embedding chỉ capture surface meaning.

Enrichment = thêm thông tin **trước khi embed** để vector representations phong phú hơn.

Sau khi chunk, **clean + augment metadata** trước khi embed.

Cleaning → better semantic matches.
Metadata → better filtering + precision.

LLM tạo **summary ngắn** cho mỗi chunk.
Embed summary thay vì (hoặc cùng với) raw chunk.
Ưu điểm: giảm noise, focus vào key info.
Dùng khi: chunks dài, nhiều filler text.

Prepend context giải thích chunk nằm ở đâu trong document.
Đã cover ở slide Contextual Embeddings.
Giảm 49% retrieval failure (alone).

LLM generate **câu hỏi mà chunk có thể trả lời**.
Index cả questions lẫn chunk → query match tốt hơn.
Ưu điểm: bridge vocabulary gap (giống HyDE nhưng offline).
Dùng khi: user queries khác ngôn ngữ với documents.

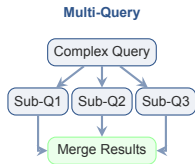
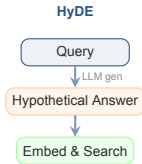
LLM extract: topic, entities, date_range, sentiment.
Gắn vào chunk metadata → enable rich filtering.
Dùng khi: corpus lớn, cần multi-faceted search.

Lưu ý: Enrichment = **one-time cost** khi indexing.
Dùng model rẻ (GPT-4o-mini, Haiku). ROI cao vì cải thiện **mọi query** sau đó.

Fix ONLINE — PreRAG

Query Transform, Corrective RAG — fix trước khi search

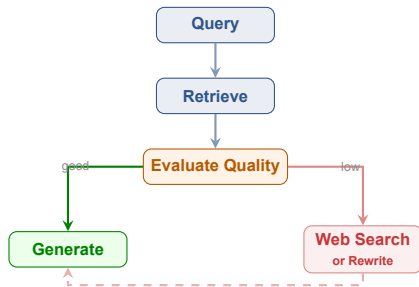
04



HyDE — Generate hypothetical answer, embed *answer* thay vì query. Bridges vocabulary gap.

Multi-Query — Decompose query thành sub-queries, retrieve mỗi cái, merge. Recall tăng cho multi-hop.

Lưu ý: HyDE tốn thêm 1 LLM call/query. Chỉ dùng khi vocabulary mismatch nghiêm trọng.



Nếu retrieval quality thấp:

1. Trigger **web search** (fallback)
2. Hoặc **query rewrite** rồi retry
3. Rồi mới generate

Tránh generate trên bad context

Route queries theo complexity:

Simple → direct LLM (no RAG)

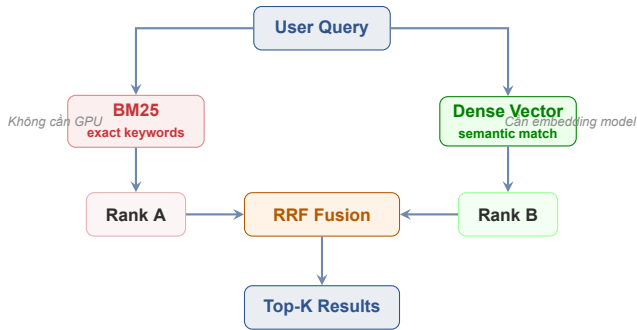
Medium → standard RAG

Complex → full pipeline + rerank

Giảm latency **40%** trung bình

Fix ONLINE — Retrieval & Augment

Hybrid Search, Metadata Filtering & Reranking — fix R và A



Merge rankings đơn giản: $\text{score}(d) = \sum \frac{1}{k + \text{rank}_i(d)}$. Không cần training, production standard.

Tiêu chí	BM25	Dense Vector	Hybrid
Exact keywords	Tốt	Yếu	Tốt
Synonyms / paraphrase	Yếu	Tốt	Tốt
Multilingual	Yếu	Tốt	Tốt
GPU required	Không	Có	Có
Latency	<5ms	~20ms	~25ms

Lưu ý: BM25 cho tiếng Việt: cần **word segmentation** (underthesea, VnCoreNLP) trước khi index. Thiếu bước này → BM25 gần **vô dụng** cho tiếng Việt.

Method	Precision	Latency	Storage	Training?
RRF (rank fusion)	Baseline	~1ms	1×	Không
Weighted Score Fusion	+2–3%	~1ms	1×	Tune weights
SPLADE (learned sparse)	+5–8%	~15ms	2×	Pre-trained
ColBERT (late interact)	+8–12%	~50ms	10×→2×*	Pre-trained
Full Tensor Fusion	+10–15%	~80ms	3×*	Fine-tune

*với compression (PLAID, quantization)

Late interaction: query tokens $\leftrightarrow$ doc tokens $\rightarrow$ MaxSim per query token.
Token-level matching chính xác hơn single-vector.
Pre-compute doc embeddings $\rightarrow$ vẫn fast retrieval.
Có thể **thay thế** cả bi-encoder + cross-encoder.

Thay thế BM25 bằng **learned sparse vectors**.
Kết hợp: exact match (BM25) + learned expansion (Dense).
Dùng inverted index $\rightarrow$ fast như BM25.
Đặc biệt tốt cho **multilingual corpus**.

Lưu ý: Tensor fusion cần labeled data. Không có
 $\rightarrow$ stick với **RRF + cross-encoder reranking**.

Gắn metadata vào mỗi chunk khi indexing.

Filter **trước** khi vector search → giảm search space, tăng precision.

Metadata phổ biến:

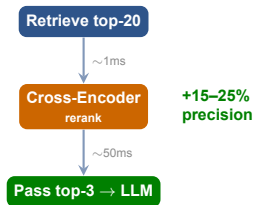
- source: tên file/URL gốc
- date: ngày tạo/cập nhật
- category: policy, FAQ, manual...
- language: ngôn ngữ
- section: chapter/heading

```
# Qdrant metadata filtering
client.search(
    collection_name="docs",
    query_vector=embedding,
    query_filter=Filter(
        must=[FieldCondition(
            key="year",
            match=MatchValue(value=2024)
        )]
    ),
    limit=20
)
```

Lưu ý: Metadata filtering phụ thuộc vector DB. Qdrant, Weaviate, Pinecone hỗ trợ tốt. Chroma hạn chế hơn.

DB	Hybrid Search	Metadata Filter	Khi nào?
Qdrant	Built-in	Rich	Default pick
Weaviate	Built-in	Rich	GraphQL fans
Pinecone	Sparse+	Good	Managed/SaaS
Milvus	Built-in	Rich	Large-scale, GPU
Neo4j	Vector+Graph	Cypher	GraphRAG
pgvector	Manual	SQL	Already Postgres
Chroma	—	Basic	Prototype
FAISS	—	—	Research only

Production: Qdrant hoặc Milvus (open-source, hybrid + metadata). Pinecone (managed, zero-ops). **GraphRAG:** Neo4j (vector + graph traversal). **Lab:** Qdrant local (Docker).



Bi: encode riêng, fast ($\sim 1\text{ms}$), no interaction
Cross: encode cùng, chậm ($\sim 50\text{ms}$), accurate hơn nhiều

Reranking Models — So sánh:

Model	Cost	Note
Cohere Rerank v3.5	API	Production default
bge-reranker-v2-m3	Free	Multilingual, tiếng Việt tốt
ms-marco-MiniLM-L-12	Free	Nhẹ, nhanh, English
Jina Reranker v2	API	Multilingual, 8K context
Flashrank	Free	Ultra-light, $< 5\text{ms}$
LLM-as-Reranker	\$\$\$	GPT-4o/Claude rerank

Retrieve **top-20** → Rerank → Keep **top-3**
→ LLM generate

Lưu ý: 30–50ms overhead đổi lấy +15–25% precision.
Highest ROI trong RAG pipeline. Tiếng Việt → bge-reranker-v2-m3.

NLI model kiểm tra **entailment** giữa query và chunk.
Filter chunks **contradict** → giảm hallucination.
Tools: cross-encoder/nli-deberta-v3-base.

Merge chunks từ **nhiều sources** (DB, web, API).
Resolve conflicts: newest wins hoặc LLM arbitrate.
Ref: MASS-RAG (multi-agent synthesis).

Gắn **source reference** vào context.
Prompt: “Cite sources using [1], [2]...”
Output: answer + citations + source links.

Chunks quá dài → **compress** trước khi LLM.
Extractive: giữ relevant sentences.
Abstractive: LLM summarize context.
Tools: LongLLMLingua, ContextualCompression.

Lưu ý: Augmentation = bước giữa Retrieval và Generation. Thường bị skip nhưng impact lớn lên answer quality + giảm token cost.

Fix ONLINE — Generate & PostRAG

Self-RAG, RAG-Fusion, Semantic Cache — fix G và validate output

Self-RAG — LLM tự quyết **khi nào retrieve**. Fine-tune model output special tokens ([Retrieve], [IsRel], [IsSup]). Không hoạt động out-of-the-box.

1. Generate **multiple query variants**
2. Retrieve cho mỗi variant → 3. RRF merge

Semantic Cache — Cache theo semantic similarity. Query mới similar >0.95 → trả cache. Giảm **30–50%** LLM calls.

Pattern	Cost	Khi nào?
HyDE	\$\$	Vocab mismatch
Multi-query	\$\$	Multi-hop Q
CRAG	\$\$	Unreliable retrieval
Self-RAG	\$\$\$	Fine-tuned model
RAG-Fusion	\$\$\$	Max recall
Sem. Cache	\$	Repeated queries

Lưu ý: Đừng dùng tất cả cùng lúc! Chọn pattern theo **failure mode cụ thể** của pipeline.

Evaluation — Đo lường RAG Pipeline

Measure first, optimize second — RAGAS, TruLens & DeepEval

Generation quality

Faithfulness

Answer claims được
context support không?

Target: ≥ 0.85

Generation quality

Answer Relevancy

Q&A cosine similarity

Target: ≥ 0.80

Context Precision

Chunks retrieved có
relevant không?

Target: ≥ 0.75

Context Recall

Đã retrieve đủ
info cần thiết chưa?

Target: ≥ 0.75

Retrieval quality

Retrieval quality

Lưu ý: RAGAS phụ thuộc judge model — scores **brittle** khi đổi judge. Luôn report judge model version cùng với scores.

Context chứa info đúng nhưng LLM bịa thêm
→ Tighten prompt (“Only use provided context”)
→ Giảm temperature, model ít hallucinate hơn
Context không chứa info cần thiết
→ Thực ra là Context Recall problem ↓

Retrieve quá nhiều irrelevant chunks
→ Thêm reranking (cross-encoder)
→ Giảm top-K, tăng similarity threshold
→ Metadata filtering

Chunks đúng tồn tại nhưng không được retrieve
→ Đổi chunking (hierarchical)
→ Thêm BM25 (hybrid search)
→ Thử HyDE hoặc Multi-Query
Chunks đúng KHÔNG tồn tại trong DB
→ Review chunking pipeline (cắt mất info)
→ Document chưa được ingest

LLM trả lời đúng nhưng không match câu hỏi
→ Improve prompt template
→ Kiểm tra context window overflow

Lưu ý: Luôn **failure analysis** (bottom-10 questions) trước khi nhìn aggregate scores. Aggregate che giấu failure patterns.

```
from ragas import evaluate
from ragas.metrics import (
    faithfulness, answer_relevancy,
    context_precision, context_recall,
)
dataset = {
    "question": questions,
    "answer": answers,
    "contexts": retrieved_chunks,
    "ground_truth": ground_truths,
}
result = evaluate(dataset,
    metrics=[faithfulness,
              answer_relevancy,
              context_precision,
              context_recall])
print(result) # DataFrame
```

Workflow chuẩn:

1. Chạy RAGAS **baseline** trước
2. Xem **bottom-5** (failure analysis)
3. Optimize theo failure mode cụ thể
4. Re-run RAGAS, so sánh

Faithfulness ≥ 0.85 , Answer Relevancy ≥ 0.80

Context Recall ≥ 0.75

Luôn failure analysis trước aggregate

Dimension	RAGAS	TruLens	DeepEval
Focus	RAG pipeline eval	Eval + Tracing (OTel)	RAG + Agents + Chatbot
Metrics	4 core metrics	RAG Triad	50+ metrics
Custom	Hạn chế	Feedback functions	G-Eval, DAG, BaseMetric
Tracing	Minimal	OpenTelemetry spans	@observe decorator
CI/CD	Manual setup	Moderate	Native Pytest
Ground truth	Không cần	Không cần	Cả hai
Setup	★ Dễ nhất	★★ Trung bình	★★ Trung bình
GitHub	~13k stars	~3k stars	~15k stars
Used by	AWS, Databricks	Snowflake, Equinix	OpenAI, Google, Microsoft

Cả 3 đều open-source, LLM-as-a-Judge, reference-free. Cost = LLM API calls cho judge model.

Khi nào:

- ✓ Focus RAG quality
- ✓ Setup nhanh 5 phút
- ✓ Team nhỏ, MVP

Không phù hợp:

- ✗ Multi-agent, chatbot
- ✗ CI/CD phức tạp

Khi nào:

- ✓ Cần tracing + eval
- ✓ Debug chính xác step fail
- ✓ Agentic multi-hop

Không phù hợp:

- ✗ Quick prototype
- ✗ Team không cần tracing

Khi nào:

- ✓ AI stack phức tạp
- ✓ CI/CD gate (Pytest)
- ✓ Safety, MCP metrics

Không phù hợp:

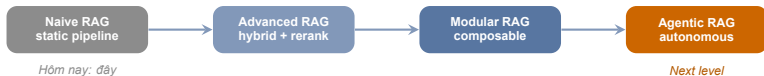
- ✗ Chỉ eval RAG đơn giản

Prototype: RAGAS (nhanh, đủ dùng) → **Debug:** +TruLens (tracing tìm bottleneck) → **CI/CD:** +DeepEval (regression gate). Các framework **không loại trừ nhau** — nhiều team dùng kết hợp.

Agentic RAG

Khi agent điều khiển RAG pipeline — từ static sang autonomous

09



Agentic RAG là gì? — Agent **tự quyết định** khi nào retrieve, query nào, bao nhiêu lần, dùng tool nào.

4 agentic patterns: **Reflection, Planning, Tool Use, Multi-Agent Collaboration.**

Ref: Singh et al., “Agentic RAG: A Survey” (2025)

Static pipeline không đủ khi:

- Query cần **multi-hop reasoning**
- Retrieval lần 1 không đủ → cần **iterative**
- Cần kết hợp **nhiều sources** (DB + web + API)
- Cần **self-correction** khi context kém

	Single-Agent	Multi-Agent	Hierarchical
Mô tả	1 agent điều phối toàn bộ retrieval + routing	Nhiều agent chuyên biệt, mỗi agent 1 data source	Agent cấp cao delegate xuống agent cấp thấp
Ưu điểm	Đơn giản, latency thấp	Scalable, parallel processing	Strategic oversight, reliable
Nhược	Không scale cho multi-domain	Coordination overhead	Latency cao, phức tạp
Khi nào	Simple QA, routing	Multi-domain synthesis	High-stakes (medical, legal)

Multi-agent synthesis: agents chuyên biệt cho **summarization, extraction, reasoning** → synthesis stage tổng hợp.
Outperform strong RAG baselines trên 4 benchmarks.

Dùng **self-knowledge** của model để filter retrieved docs.
RL-based training → model biết “mình biết gì, không biết gì”.
Giảm input documents + tăng generation quality.

Demo & Thực hành

Naive RAG vs Production RAG — head-to-head

08

1. **Pipeline A** (naive): fixed chunks 512 + dense-only → chạy RAGAS
2. **Pipeline B** (production): hierarchical chunks + hybrid search + Cohere Rerank → chạy RAGAS
3. **Pipeline C** (bonus): thêm contextual embeddings → so sánh thêm
4. **Failure analysis**: zoom vào bottom-5 questions — dùng Diagnostic Tree để map failure → pipeline step

Mục tiêu: Build Production RAG Pipeline

Deliverable: RAG pipeline đạt $RAGAS \geq 0.75$ + deploy với LangServe: API endpoint + streaming response

Thời gian: 2 giờ

Các bước:

1. **Chunking pipeline:** semantic + hierarchical — A/B test vs fixed-size
2. **Enrichment:** thử contextual prepend hoặc HyQA cho subset
3. **Hybrid search:** BM25 (Vietnamese segmentation) + Dense + RRF
4. **Reranking:** bge-reranker-v2-m3 hoặc Cohere Rerank — top-20 → top-3
5. **RAGAS evaluation:** test set 50+ questions, target $F \geq 0.85$, $CR \geq 0.75$

Rubric:

Tiêu chí	Điểm
Chunking A/B test + so sánh	20
Hybrid search (BM25 + Dense)	15
Vietnamese segmentation (BM25)	5
Reranking integrated	15
RAGAS eval + report	25
Failure analysis bottom-10	15
Code quality + docs	5
Tổng	100

requirements.txt (pinned versions)
Skeleton notebook + sample test set (20 Q&A)
Docker compose cho Qdrant local

GitHub repo + RAGAS report +
failure analysis + latency break-
down

Embedding 1M chunks $\times$ \$0.02/1M tokens $\approx$ \$10–50
Contextual embeddings: +\$50–200 (GPT-4o-mini)
Vector DB storage: $\sim$ \$20–50/month (Qdrant Cloud)

Naive RAG: $\sim$ \$1,500/month
Production RAG: $\sim$ \$2,200/month (+47%)

Accuracy 85% vs 60% $\rightarrow$ ít escalation
ROI dương sau 2–3 tháng

Dùng GPT-4o: chi phí $\times$ 10–15.

Embedding: $\sim$ \$0.00002 · Reranking: $\sim$ \$0.001
LLM generation: $\sim$ \$0.01–0.05
Total: $\sim$ \$0.01–0.06/query

Lưu ý: Semantic caching giảm 30–50% LLM calls $\rightarrow$ tiết kiệm đáng kể khi nhiều user hỏi tương tự.